Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: C

Instructor Name:

Course: Operating Systems Lab

Program: (BS) - Computer Science

Lab #01

Task 1: List all files in the current directory.

```
faisal152@faisal152:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

Task 2: List all files in human readable form.

```
faisal152@faisal152:~$ ls -lh
total 32K
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Desktop
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Documents
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Downloads
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Music
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Pictures
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Public
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Templates
drwxr-xr-x 2 faisal152 faisal152 4.0K May  8 13:39 Videos
```

Task 3: Display your current directory.

```
faisal152@faisal152:~$ pwd
/home/faisal152
```

Task 4: Change to the / directory.

```
faisal152@faisal152:~$ cd /
faisal152@faisal152:/$
```

Task 5: Go to the ~ directory from the root directory.

```
faisal152@faisal152:/$ cd ~
faisal152@faisal152:~$
```

Task 6: List the contents of the root directory.

```
faisal152@faisal152:~$ ls /
bin   cdrom  etc   lib    lib64   lost+found  mnt  proc  run   snap  swapfile  tmp  var
boot  dev    home  lib32  libx32  media       opt  root  sbin  srv   sys       usr
```

Task 7: List the files in /boot in a human readable format.

```
faisal152@faisal152:~$ ls -lh /boot
total 89M
-rw-r--r-- 1 root root 257K Feb 22  2023 config-5.15.0-67-generic
drwx------ 2 root root 4.0K Jan  1  1970 efi
drwxr-xr-x 4 root root 4.0K May  8 13:33 grub
lrwxrwxrwx 1 root root   28 May  8 13:32 initrd.img -> initrd.img-5.15.0-67-generic
-rw-r--r-- 1 root root  71M May  8 13:36 initrd.img-5.15.0-67-generic
lrwxrwxrwx 1 root root   28 May  8 13:26 initrd.img.old -> initrd.img-5.15.0-67-generic
-rw-r--r-- 1 root root 179K Aug 18  2020 memtest86+.bin
-rw-r--r-- 1 root root 181K Aug 18  2020 memtest86+.elf
-rw-r--r-- 1 root root 181K Aug 18  2020 memtest86+_multiboot.bin
-rw------- 1 root root 6.0M Feb 22  2023 System.map-5.15.0-67-generic
lrwxrwxrwx 1 root root   25 May  8 13:32 vmlinuz -> vmlinuz-5.15.0-67-generic
-rw-r--r-- 1 root root  11M Mar 16  2023 vmlinuz-5.15.0-67-generic
```

Task 8: Create a directory testdir in your ~ directory.

```
faisal152@faisal152:~$ mkdir ~/testdir
faisal152@faisal152:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  testdir  Tests  Videos
```

Task 9: Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).

```
faisal152@faisal152:~$ mkdir -p ~/dir1/dir2/dir3
faisal152@faisal152:~$ ls
Desktop  dir1  Documents  Downloads  Music  Pictures  Public  Templates  testdir  Tests  Videos
```

Task 10: Remove the directory testdir.

```
faisal152@faisal152:~$ rm -r ~/testdir
faisal152@faisal152:~$ ls
Desktop  dir1  Documents  Downloads  Music  Pictures  Public  Templates  Tests  Videos
```

Task 11: Move TesingDir from ~ to ~/Tests directories.

```
faisal152@faisal152:~$ mkdir ~/TesingDir
faisal152@faisal152:~$ mkdir ~/Tests
faisal152@faisal152:~$ ls
Desktop  dir1  Documents  Downloads  Music  Pictures  Public  Templates  TesingDir  Tests  Videos
faisal152@faisal152:~$ mv ~/TesingDir ~/Tests
faisal152@faisal152:~$ ls
Desktop  dir1  Documents  Downloads  Music  Pictures  Public  Templates  Tests  Videos
```

Task 12: Rename a directory from TesingDir to testingDir

```
faisal152@faisal152:~$ cd ~/Tests
```

```
faisal152@faisal152:~/Tests$ ls
TesingDir
faisal152@faisal152:~/Tests$ mv ~/Tests/TesingDir ~/Tests/testingDir
faisal152@faisal152:~/Tests$ ls
testingDir
```

Lab #02

Task 1: Use ls to output the contents of the /etc/ directory to a file called etc.txt.

```
faisal152@faisal152:~$ ls /etc/ > etc.txt
faisal152@faisal152:~$
```

Task 2: Make a file bioData.txt and check that it contains your name using grep,also use i & v flags.

```
faisal152@faisal152:~$ cat > biodata.txt
Faisal
152
cs
faisal152@faisal152:~$ grep -i faisal biodata.txt
Faisal
```

```
faisal152@faisal152:~$ grep -i -v faisal biodata.txt
152
cs
```

Task 3: Append information in bioData.txt.

```
faisal152@faisal152:~$ cat >> biodata.txt
Sir Syed University
faisal152@faisal152:~$ cat < biodata.txt
Faisal
152
cs
Sir Syed University
```

Task 4: Sort the file number.txt using flags.

```
faisal152@faisal152:~$ cat > number.txt
3
1
7
2
9
8
5
```

```
faisal152@faisal152:~$ sort -n number.txt
1
2
3
5
7
8
9
```

Task 5: Show all the information like total lines, total words and total character inside of your file.

```
faisal152@faisal152:~$ wc biodata.txt
 4  6 34 biodata.txt
```

Task 6: Change case of the letters of your file bioData.txt.

```
faisal152@faisal152:~$ tr '[:lower:]' '[:upper:]' < biodata.txt > biodata_upper.txt
faisal152@faisal152:~$ cat < biodata_upper.txt
FAISAL
152
CS
SIR SYED UNIVERSITY
```

Task 7: Implement stderr (2> and 2>&1).

```
faisal152@faisal152:~$ ls xyz.txt 2> error.txt
faisal152@faisal152:~$ cat < error.txt
ls: cannot access 'xyz.txt': No such file or directory
```

```
faisal152@faisal152:~$ ls xyz.txt > output.txt 2>&1
faisal152@faisal152:~$ cat < output.txt
ls: cannot access 'xyz.txt': No such file or directory
```

Task 8: Show last 5 data from numbers.txt

```
faisal152@faisal152:~$ tail -n 5 number.txt
7
2
9
8
5
```

Task 9: Count all duplicates in your file bioData.txt

```
faisal152@faisal152:~$ sort biodata.txt | uniq -c
      2 152
      2 cs
      1 Faisal
      1 Sir Syed University
```

Task 10: Show your os version.

```
faisal152@faisal152:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.6 LTS"
VERSION_ID="20.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=focal
UBUNTU_CODENAME=focal
```

Task 11: Compress the file bioData.txt using gzip and bzip2 algorithm.

```
faisal152@faisal152:~$ gzip -c biodata.txt > biodata.txt.gz
faisal152@faisal152:~$ bzip2 -k biodata.txt
```

Task 12: Show contents of the file bioData.txt.gz without decompressing.

```
faisal152@faisal152:~$ zcat biodata.txt.gz
Faisal
152
cs
Sir Syed University
152
cs
```

Lab #03

Task 1: Create two groups webProject and SQA.

```
faisal152@faisal152:~$ sudo groupadd webproject
[sudo] password for faisal152:
faisal152@faisal152:~$ sudo groupadd SQA
```

Task 2: Create two users , user1 and user2 and add them in a group webProject and SQA respectively.

```
faisal152@faisal152:~$ sudo useradd -G webproject user1
faisal152@faisal152:~$ sudo useradd -G SQA user2
```

Task 3: Show All users.

```
faisal152@faisal152:~$ tail -5 /etc/passwd
sssd:x:127:132:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
faisal152:x:1000:1000:Muhammad Faisal,,,:/home/faisal152:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
user1:x:1001:1003::/home/user1:/bin/sh
user2:x:1002:1004::/home/user2:/bin/sh
```

Task 4: Show All groups.

```
faisal152@faisal152:~$ tail -5 /etc/group
systemd-coredump:x:999:
webproject:x:1001:user1
SQA:x:1002:user2
user1:x:1003:
user2:x:1004:
```

Task 5: Set user1 as group member of SQA.

```
faisal152@faisal152:~$ sudo usermod -aG SQA user1
[sudo] password for faisal152:
```

Task 6: Switch to user2.

```
faisal152@faisal152:~$ su - user2
```

Task 7: Create a filename OSLabFile.txt and change its permission , read and execute only in group & user.

```
faisal152@faisal152:~$ touch OSLabFile.txt
faisal152@faisal152:~$ chmod 550 OSLabFile.txt
```

Task 8: Delete user1

```
faisal152@faisal152:~$ sudo userdel -r user1
[sudo] password for faisal152:
userdel: user1 mail spool (/var/mail/user1) not found
userdel: user1 home directory (/home/user1) not found
```

Task 9: Delete SQA.

```
faisal152@faisal152:~$ sudo groupdel SQA
```

Task 10: Change password of user1.

```
faisal152@faisal152:~$ sudo useradd user1
```

```
faisal152@faisal152:~$ sudo passwd user1
New password:
Retype new password:
passwd: password updated successfully
```

Lab #04

Task 1: Create a MyInfo.txt using vi editor.

```
faisal152@faisal152:~$ vi MYinfo.txt
```

Task 2: Write Your name , roll number and section in MyInfo.txt

```
Faisal
152
C
~
```

Task 3: Save the File.

```
~
:wq
```

```
Faisal
152
C
~
~
```

```
~
"MYinfo.txt" 3 lines, 13 characters
```

Task 4: Delete your section from MyInfo.txt.

```
Faisal
152
~
```

Task 5: Show content of MyInfo.txt using cat.

```
faisal152@faisal152:~$ cat MYinfo.txt
Faisal
152
```

Task 6: Print host name

```
faisal152@faisal152:~$ hostname
faisal152
```

Task 7: Print your(system) user name.

```
faisal152@faisal152:~$ whoami
faisal152
```

Task 8: Show the locatoin where ls ,pwd ,rmdir & cat source code is present.

```
faisal152@faisal152:~$ which ls
/usr/bin/ls
```

```
faisal152@faisal152:~$ which pwd
/usr/bin/pwd
```

```
faisal152@faisal152:~$ which rmdir
/usr/bin/rmdir
```

```
faisal152@faisal152:~$ which cat
/usr/bin/cat
```

Lab #05

Task 1: Create a file OSLabFile5.1 with date & time 25th January, 2025 3:00 PM.

```
faisal152@faisal152:~$ touch -t 202501251500 OSLabFile5.1
```

Task 2: Find the binary and source file location of “ls” command

- Finding the Binary Location

```
faisal152@faisal152:~$ which ls
/usr/bin/ls
```

- Finding the Source file Location

```
faisal152@faisal152:~$ sudo apt-get install apt-src
faisal152@faisal152:~$ sudo apt-get update
faisal152@faisal152:~$ apt-get source coreutils
faisal152@faisal152:~$ cd coreutils-*/src
faisal152@faisal152:~/coreutils-8.30/src$
```

Task 3: Creates 5 threads each thread prints a "Hello World!" message, and then terminates.

```
faisal152@faisal152:~$ gedit faisal_threads.c
```

Open faisal_threads.c

```c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5

// Function to be executed by each thread
void *printHello(void *threadID) {
    long tid = (long)threadID;
    printf("Thread #%ld: Hello World!\n", tid);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;

    // Create threads
    for (t = 0; t < NUM_THREADS; t++) {
        printf("Creating thread #%ld\n", t);
        rc = pthread_create(&threads[t], NULL, printHello, (void *)t);
        if (rc) {
            printf("Error: Unable to create thread #%ld\n", t);
            return -1;
        }
    }

    // Join threads
    for (t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    return 0;
}
```

```
faisal152@faisal152:~$ gedit faisal_threads.c
faisal152@faisal152:~$ chmod +x faisal_threads.c
faisal152@faisal152:~$ gcc -o faisal_threads faisal_threads.c -lpthread
faisal152@faisal152:~$ ./faisal_threads
Creating thread #0
Creating thread #1
Creating thread #2
Creating thread #3
Creating thread #4
Thread #0: Hello World!
Thread #1: Hello World!
Thread #2: Hello World!
Thread #3: Hello World!
Thread #4: Hello World!
```

Lab #06

Task 1. For below processes, write FCFS and SJF processes to calculate waiting time for each process and average waiting time.

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

```
faisal152@faisal152:~$ dotnet new console -o SchedullingApp

The template "Console App" was created successfully.
```

```
faisal152@faisal152:~$ cd SchedullingApp
faisal152@faisal152:~/SchedullingApp$
```

FCFS

```
faisal152@faisal152:~/schedullingApp$ nano fcfs.cs
```

```
using System;
using System.Collections.Generic;

public class Process
{
    public int Id { get; set; }
    public int ArrivalTime { get; set; }
    public int BurstTime { get; set; }
    public int OriginalBurstTime { get; set; }
    public int WaitingTime { get; set; }
    public int TurnaroundTime { get; set; }

    public Process(int id, int arrivalTime, int burstTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
        BurstTime = burstTime;
        OriginalBurstTime = burstTime;
        WaitingTime = 0;
        TurnaroundTime = 0;
    }
}

public class FCFS
{
    public List<Process> Processes { get; set; }

    public FCFS(List<Process> processes)
    {
        Processes = processes;
    }

    public void Schedule()
    {
        int currentTime = 0;
        int n = Processes.Count;

        // Sort processes by arrival time (though FCFS doesn't strictly require this)
        Processes.Sort((a, b) => a.ArrivalTime.CompareTo(b.ArrivalTime));
```

```
        foreach (var proc in Processes)
        {
            // Process each job in arrival order
            if (proc.ArrivalTime > currentTime)
            {
                currentTime = proc.ArrivalTime;
            }

            proc.WaitingTime = currentTime - proc.ArrivalTime;
            proc.TurnaroundTime = proc.WaitingTime + proc.BurstTime;
            currentTime += proc.BurstTime;
        }
    }

    public void DisplayResults()
    {
        int totalWaitingTime = 0;
        int totalTurnaroundTime = 0;

        Console.WriteLine();
        Console.WriteLine("Process\tArrival\tBurst\tWaiting\tTurnaround");
        foreach (var proc in Processes)
        {

Console.WriteLine($"{proc.Id}\t{proc.ArrivalTime}\t{proc.OriginalBurstTime}\t{proc.WaitingTime}\t{p
roc.TurnaroundTime}");
            totalWaitingTime += proc.WaitingTime;
            totalTurnaroundTime += proc.TurnaroundTime;
        }

        float averageWaitingTime = (float)totalWaitingTime / Processes.Count;
        float averageTurnaroundTime = (float)totalTurnaroundTime / Processes.Count;

        Console.WriteLine();
        Console.WriteLine($"Average Waiting Time: {averageWaitingTime:F2}");
        Console.WriteLine($"Average Turnaround Time: {averageTurnaroundTime:F2}");
    }
}

public class Program
{
    public static void Main()
    {
        List<Process> processes = new List<Process>();

        Console.Write("Enter the number of processes: ");
        int numberOfProcesses = Convert.ToInt32(Console.ReadLine());

        for (int i = 1; i <= numberOfProcesses; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i}: ");
            int arrivalTime = Convert.ToInt32(Console.ReadLine());
            Console.Write($"Enter burst time for process {i}: ");
            int burstTime = Convert.ToInt32(Console.ReadLine());

            processes.Add(new Process(i, arrivalTime, burstTime));
        }

        // Create an FCFS scheduler
        FCFS fcfs = new FCFS(processes);

        // Schedule the processes
        fcfs.Schedule();

        // Display the results
        fcfs.DisplayResults();

        Console.ReadLine();
```

```
    }
}
```

```
faisal152@faisal152:~/SchedullingApp$ dotnet build
```

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 4

Enter arrival time for process 1: 0
Enter burst time for process 1: 5

Enter arrival time for process 2: 1
Enter burst time for process 2: 3

Enter arrival time for process 3: 2
Enter burst time for process 3: 8

Enter arrival time for process 4: 3
Enter burst time for process 4: 6

Process Arrival Burst   Waiting Turnaround
1       0       5       0       5
2       1       3       4       7
3       2       8       6       14
4       3       6       13      19

Average Waiting Time: 5.75
```

SJF

```
faisal152@faisal152:~/schedullingApp$ nano SJF.cs
```

```
using System;
using System.Collections.Generic;

public class Process
{
    public int Id { get; set; }
    public int ArrivalTime { get; set; }
    public int BurstTime { get; set; }
    public int OriginalBurstTime { get; set; }
    public int WaitingTime { get; set; }
    public int TurnaroundTime { get; set; }

    public Process(int id, int arrivalTime, int burstTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
        BurstTime = burstTime;
        OriginalBurstTime = burstTime;
        WaitingTime = 0;
        TurnaroundTime = 0;
    }
}

public class ShortestJobFirst
{
    public List<Process> Processes { get; set; }

    public ShortestJobFirst(List<Process> processes)
    {
        Processes = processes;
```

```
    }

    public void Schedule()
    {
        // Sort processes based on arrival time, then burst time
        Processes.Sort((a, b) => {
            int compareArrival = a.ArrivalTime.CompareTo(b.ArrivalTime);
            if (compareArrival == 0)
            {
                return a.BurstTime.CompareTo(b.BurstTime);
            }
            return compareArrival;
        });

        int currentTime = 0;
        int completedProcesses = 0;
        int n = Processes.Count;

        while (completedProcesses < n)
        {
            // Find the next shortest job that has arrived
            Process shortestProcess = null;
            for (int i = 0; i < n; i++)
            {
                Process proc = Processes[i];
                if (proc.ArrivalTime <= currentTime && proc.BurstTime > 0)
                {
                    if (shortestProcess == null || proc.BurstTime < shortestProcess.BurstTime)
                    {
                        shortestProcess = proc;
                    }
                }
            }

            if (shortestProcess != null)
            {
                // Process the shortest process found
                currentTime += shortestProcess.BurstTime;
                shortestProcess.TurnaroundTime = currentTime - shortestProcess.ArrivalTime;
                shortestProcess.WaitingTime = shortestProcess.TurnaroundTime -
shortestProcess.OriginalBurstTime;

                // Mark the process as completed
                shortestProcess.BurstTime = 0;
                completedProcesses++;
            }
            else
            {
                // If no process found to execute, just increment the time
                currentTime++;
            }
        }
    }

    public void DisplayResults()
    {
        int totalWaitingTime = 0;
        int totalTurnaroundTime = 0;

        Console.WriteLine();
        Console.WriteLine("Process\tArrival\tBurst\tWaiting\tTurnaround");
        foreach (var proc in Processes)
        {

Console.WriteLine($"{proc.Id}\t{proc.ArrivalTime}\t{proc.OriginalBurstTime}\t{proc.WaitingTime}\t{p
roc.TurnaroundTime}");
            totalWaitingTime += proc.WaitingTime;
            totalTurnaroundTime += proc.TurnaroundTime;
        }
```

```
        float averageWaitingTime = (float)totalWaitingTime / Processes.Count;
        float averageTurnaroundTime = (float)totalTurnaroundTime / Processes.Count;

        Console.WriteLine();
        Console.WriteLine($"Average Waiting Time: {averageWaitingTime:F2}");
        Console.WriteLine($"Average Turnaround Time: {averageTurnaroundTime:F2}");
    }
}

public class Program
{
    public static void Main()
    {
        List<Process> processes = new List<Process>();

        Console.Write("Enter the number of processes: ");
        int numberOfProcesses = Convert.ToInt32(Console.ReadLine());

        for (int i = 1; i <= numberOfProcesses; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i}: ");
            int arrivalTime = Convert.ToInt32(Console.ReadLine());
            Console.Write($"Enter burst time for process {i}: ");
            int burstTime = Convert.ToInt32(Console.ReadLine());

            processes.Add(new Process(i, arrivalTime, burstTime));
        }

        // Create a SJF scheduler
        ShortestJobFirst sjf = new ShortestJobFirst(processes);

        // Schedule the processes
        sjf.Schedule();

        // Display the results
        sjf.DisplayResults();

        Console.ReadLine();
    }
}
```

```
faisal152@faisal152:~/SchedullingApp$ dotnet build
```

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 4

Enter arrival time for process 1: 0
Enter burst time for process 1: 5

Enter arrival time for process 2: 1
Enter burst time for process 2: 3

Enter arrival time for process 3: 2
Enter burst time for process 3: 8

Enter arrival time for process 4: 3
Enter burst time for process 4: 6

Process Arrival Burst   Waiting Turnaround
1       0       5       0       5
2       1       3       4       7
3       2       8       12      20
4       3       6       5       11

Average Waiting Time: 5.25
Average Turnaround Time: 10.75
```

Task 2: For below processes, write FCFS and SJF processes to calculate waiting time for each process and average waiting time.

Process	Arrival Time	Execute Time
P0	2	6
P1	5	2
P2	1	8
P3	0	3
P4	4	4

FCFS

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 5

Enter arrival time for process 1: 2
Enter burst time for process 1: 6

Enter arrival time for process 2: 5
Enter burst time for process 2: 2

Enter arrival time for process 3: 1
Enter burst time for process 3: 8

Enter arrival time for process 4: 0
Enter burst time for process 4: 3

Enter arrival time for process 5: 4
Enter burst time for process 5: 4

Process Arrival Burst   Waiting Turnaround
4       0       3       0       3
3       1       8       2       10
1       2       6       9       15
5       4       4       13      17
2       5       2       16      18

Average Waiting Time: 8.00
Average Turnaround Time: 12.60
```

SJF

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 5

Enter arrival time for process 1: 2
Enter burst time for process 1: 6

Enter arrival time for process 2: 5
Enter burst time for process 2: 2

Enter arrival time for process 3: 1
Enter burst time for process 3: 8

Enter arrival time for process 4: 0
Enter burst time for process 4: 3

Enter arrival time for process 5: 4
Enter burst time for process 5: 4

Process Arrival Burst   Waiting Turnaround
4       0       3       0       3
3       1       8       14      22
1       2       6       1       7
5       4       4       7       11
2       5       2       4       6

Average Waiting Time: 5.20
```

Lab #07

Task 1: Perform the above code with 8 Number of processes by randomly setting the arrival time and burst time and priority.
(a) Use Round Robin Algorithm
(b) Use Priority Based Algorithm

(a)

```
faisal152@faisal152:~$ dotnet new console -o SchedullingApp

The template "Console App" was created successfully.
```

```
faisal152@faisal152:~$ cd SchedullingApp
faisal152@faisal152:~/SchedullingApp$
```

```
faisal152@faisal152:~/schedullingApp$ nano RoundRobin.cs
```

```
using System;

class GFG
{
    public static void roundRobin(String[] p, int[] a, int[] b, int n)
    {
        // Result of average times
        int res = 0;
        int resc = 0;

        // For sequence storage
        String seq = "";

        // Copy the burst array and arrival array for not affecting the actual array
        int[] res_b = new int[b.Length];
        int[] res_a = new int[a.Length];

        for (int i = 0; i < res_b.Length; i++)
        {
            res_b[i] = b[i];
            res_a[i] = a[i];
        }

        // Critical time of system
        int t = 0;

        // For store the waiting time
        int[] w = new int[p.Length];

        // For store the Completion time
        int[] comp = new int[p.Length];

        while (true)
        {
            bool flag = true;
            for (int i = 0; i < p.Length; i++)
            {
                // These conditions are for if arrival is not on zero

                // Check that if they come before quantum time
                if (res_a[i] <= t)
                {
                    if (res_a[i] <= n)
                    {
                        if (res_b[i] > 0)
                        {
                            flag = false;
```

```
                    if (res_b[i] > n)
                    {
                        // Make decrease the burst time
                        t = t + n;
                        res_b[i] = res_b[i] - n;
                        res_a[i] = res_a[i] + n;
                        seq += "->" + p[i];
                    }
                    else
                    {
                        // For last time
                        t = t + res_b[i];

                        // Store completion time
                        comp[i] = t - a[i];

                        // Store waiting time
                        w[i] = t - b[i] - a[i];
                        res_b[i] = 0;

                        // Add sequence
                        seq += "->" + p[i];
                    }
                }
            }
            else if (res_a[i] > n)
            {
                // If any process has less arrival time than the current process, execute
them
                for (int j = 0; j < p.Length; j++)
                {
                    // Compare
                    if (res_a[j] < res_a[i])
                    {
                        if (res_b[j] > 0)
                        {
                            flag = false;
                            if (res_b[j] > n)
                            {
                                t = t + n;
                                res_b[j] = res_b[j] - n;
                                res_a[j] = res_a[j] + n;
                                seq += "->" + p[j];
                            }
                            else
                            {
                                t = t + res_b[j];
                                comp[j] = t - a[j];
                                w[j] = t - b[j] - a[j];
                                res_b[j] = 0;
                                seq += "->" + p[j];
                            }
                        }
                    }
                }

                // Now the previous process according to ith process
                if (res_b[i] > 0)
                {
                    flag = false;

                    // Check for greater burst times
                    if (res_b[i] > n)
                    {
                        t = t + n;
                        res_b[i] = res_b[i] - n;
                        res_a[i] = res_a[i] + n;
                        seq += "->" + p[i];
                    }
```

```
                        else
                        {
                            t = t + res_b[i];
                            comp[i] = t - a[i];
                            w[i] = t - b[i] - a[i];
                            res_b[i] = 0;
                            seq += "->" + p[i];
                        }
                    }
                }
            }
            else if (res_a[i] > t)
            {
                t++;
                i--;
            }
        }

        // Exit the while loop
        if (flag)
        {
            break;
        }
    }

    Console.WriteLine();
    Console.WriteLine("process\tctime\twtime");
    for (int i = 0; i < p.Length; i++)
    {
        Console.WriteLine(" " + p[i] + "\t" + comp[i] + "\t" + w[i]);

        res = res + w[i];
        resc = resc + comp[i];
    }

    Console.WriteLine();
    Console.WriteLine("Average waiting time is " + (float)res / p.Length);
    Console.WriteLine("Average completion time is " + (float)resc / p.Length);
    Console.WriteLine("Sequence is like that " + seq);
}

// Driver Code
public static void Main(String[] args)
{
    // Get the number of processes from the user
    Console.Write("Enter the number of processes: ");
    int numProcesses = int.Parse(Console.ReadLine());

    // Name of the processes
    String[] name = new String[numProcesses];
    for (int i = 0; i < numProcesses; i++)
    {
        name[i] = "p" + (i + 1);
    }

    Console.WriteLine();
    // Arrival time for every process
    int[] arrivaltime = new int[numProcesses];
    Console.WriteLine("Enter the arrival times:");
    for (int i = 0; i < numProcesses; i++)
    {
        Console.Write("Arrival time for " + name[i] + ": ");
        arrivaltime[i] = int.Parse(Console.ReadLine());
    }

    Console.WriteLine();
    // Burst time for every process
    int[] bursttime = new int[numProcesses];
    Console.WriteLine("Enter the burst times:");
```

```
        for (int i = 0; i < numProcesses; i++)
        {
            Console.Write("Burst time for " + name[i] + ": ");
            bursttime[i] = int.Parse(Console.ReadLine());
        }

        Console.WriteLine();
        // Quantum time of each process
        Console.Write("Enter the quantum time: ");
        int q = int.Parse(Console.ReadLine());

        // Call the function for output
        roundRobin(name, arrivaltime, bursttime, q);
        Console.ReadLine();
    }
}
```

```
faisal152@faisal152:~/SchedullingApp$ dotnet build
```

```
faisal152@faisal152:~/SchedullingApp$ dotnet run
```

```
Enter the number of processes: 8

Enter the arrival times:
Arrival time for p1: 1
Arrival time for p2: 2
Arrival time for p3: 3
Arrival time for p4: 4
Arrival time for p5: 5
Arrival time for p6: 6
Arrival time for p7: 7
Arrival time for p8: 8

Enter the burst times:
Burst time for p1: 2
Burst time for p2: 4
Burst time for p3: 6
Burst time for p4: 8
Burst time for p5: 1
Burst time for p6: 9
Burst time for p7: 2
Burst time for p8: 4

Enter the quantum time: 2
```

```
process ctime   wtime
 p1     2       0
 p2     9       5
 p3     19      13
 p4     28      20
 p5     7       6
 p6     31      22
 p7     13      11
 p8     22      18

Average waiting time is 11.875
Average completion time is 16.375
Sequence is like that ->p1->p2->p3->p4->p2->p5->p3->p6->p4->p7->p3->p8->p4->p6->p8->p4->p6->p6->p6
```

(b)

```
faisal152@faisal152:~/schedullingApp$ nano Priority.cs
faisal152@faisal152:~/SchedullingApp$ dotnet build
```

```
using System;
using System.Collections.Generic;

public class Process
{
    public int Id { get; set; }
    public int ArrivalTime { get; set; }
    public int BurstTime { get; set; }
    public int OriginalBurstTime { get; set; }
    public int WaitingTime { get; set; }
    public int TurnaroundTime { get; set; }

    public Process(int id, int arrivalTime, int burstTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
        BurstTime = burstTime;
        OriginalBurstTime = burstTime;
        WaitingTime = 0;
        TurnaroundTime = 0;
    }
}

public class ShortestJobFirst
{
    public List<Process> Processes { get; set; }

    public ShortestJobFirst(List<Process> processes)
    {
        Processes = processes;
    }

    public void Schedule()
    {
        // Sort processes based on arrival time, then burst time
        Processes.Sort((a, b) => {
            int compareArrival = a.ArrivalTime.CompareTo(b.ArrivalTime);
            if (compareArrival == 0)
            {
                return a.BurstTime.CompareTo(b.BurstTime);
            }
            return compareArrival;
        });

        int currentTime = 0;
        int completedProcesses = 0;
        int n = Processes.Count;

        while (completedProcesses < n)
        {
            // Find the next shortest job that has arrived
            Process shortestProcess = null;
            for (int i = 0; i < n; i++)
            {
                Process proc = Processes[i];
                if (proc.ArrivalTime <= currentTime && proc.BurstTime > 0)
                {
                    if (shortestProcess == null || proc.BurstTime < shortestProcess.BurstTime)
                    {
                        shortestProcess = proc;
                    }
                }
```

```
            }

            if (shortestProcess != null)
            {
                // Process the shortest process found
                currentTime += shortestProcess.BurstTime;
                shortestProcess.TurnaroundTime = currentTime - shortestProcess.ArrivalTime;
                shortestProcess.WaitingTime = shortestProcess.TurnaroundTime -
shortestProcess.OriginalBurstTime;

                // Mark the process as completed
                shortestProcess.BurstTime = 0;
                completedProcesses++;
            }
            else
            {
                // If no process found to execute, just increment the time
                currentTime++;
            }
        }
    }

    public void DisplayResults()
    {
        Console.WriteLine();
        Console.WriteLine("Process\tArrival\tBurst\tWaiting\tTurnaround");
        foreach (var proc in Processes)
        {

Console.WriteLine($"{proc.Id}\t{proc.ArrivalTime}\t{proc.OriginalBurstTime}\t{proc.WaitingTime}\t{p
roc.TurnaroundTime}");
        }
    }
}

public class Program
{
    public static void Main()
    {
        List<Process> processes = new List<Process>();

        Console.Write("Enter the number of processes: ");
        int numberOfProcesses = Convert.ToInt32(Console.ReadLine());

        for (int i = 1; i <= numberOfProcesses; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i}: ");
            int arrivalTime = Convert.ToInt32(Console.ReadLine());
            Console.Write($"Enter burst time for process {i}: ");
            int burstTime = Convert.ToInt32(Console.ReadLine());

            processes.Add(new Process(i, arrivalTime, burstTime));
        }

        // Create a SJF scheduler
        ShortestJobFirst sjf = new ShortestJobFirst(processes);

        // Schedule the processes
        sjf.Schedule();

        // Display the results
        sjf.DisplayResults();

        Console.ReadLine();
    }
}
using System;
```

```
class Program
{
    static int totalprocess;
    static int[][] proc;
    static int[] arrivaltime;
    static int[] bursttime;
    static int[] priority;

    // Driver code
    static void Main(string[] args)
    {
        // Get the number of processes from the user
        Console.Write("Enter the number of processes: ");
        totalprocess = int.Parse(Console.ReadLine());

        proc = new int[totalprocess][];
        arrivaltime = new int[totalprocess];
        bursttime = new int[totalprocess];
        priority = new int[totalprocess];

        // Get arrival time, burst time, and priority for each process
        for (int i = 0; i < totalprocess; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i + 1}: ");
            arrivaltime[i] = int.Parse(Console.ReadLine());

            Console.Write($"Enter burst time for process {i + 1}: ");
            bursttime[i] = int.Parse(Console.ReadLine());

            Console.Write($"Enter priority for process {i + 1}: ");
            priority[i] = int.Parse(Console.ReadLine());
        }

        // Initialize the proc array
        for (int i = 0; i < totalprocess; i++)
        {

            proc[i] = new int[4];
            proc[i][0] = arrivaltime[i];
            proc[i][1] = bursttime[i];
            proc[i][2] = priority[i];
            proc[i][3] = i + 1;
        }

        // Sort by priority and then by arrival time
        Array.Sort(proc, (x, y) => x[2].CompareTo(y[2]));
        Array.Sort(proc, (x, y) => x[0].CompareTo(y[0]));

        Findgc();
        Console.ReadLine();
    }

    // Using FCFS Algorithm to find Waiting time
    static void GetWtTime(int[] wt)
    {
        // Declaring service array that stores cumulative burst time
        int[] service = new int[totalprocess];

        // Initialising initial elements of the arrays
        service[0] = 0;
        wt[0] = 0;

        for (int i = 1; i < totalprocess; i++)
        {
            service[i] = proc[i - 1][1] + service[i - 1];
            wt[i] = service[i] - proc[i][0] + 1;

            // If waiting time is negative, change it to zero
```

```
            if (wt[i] < 0)
            {
                wt[i] = 0;
            }
        }
    }

    // Filling turnaround time array
    static void GetTatTime(int[] tat, int[] wt)
    {
        for (int i = 0; i < totalprocess; i++)
        {
            tat[i] = proc[i][1] + wt[i];
        }
    }

    static void Findgc()
    {
        // Declare waiting time and turnaround time array
        int[] wt = new int[totalprocess];
        int[] tat = new int[totalprocess];
        int wavg = 0;
        int tavg = 0;

        // Function call to find waiting time array
        GetWtTime(wt);

        // Function call to find turnaround time
        GetTatTime(tat, wt);

        int[] stime = new int[totalprocess];
        int[] ctime = new int[totalprocess];
        stime[0] = 1;
        ctime[0] = stime[0] + tat[0];

        Console.WriteLine();
        Console.WriteLine("Process_no\tStart_time\tComplete_time\tTurn_Around_Time\tWaiting_Time");

        // Calculating starting and ending time
        for (int i = 0; i < totalprocess; i++)
        {
            wavg += wt[i];
            tavg += tat[i];
            Console.WriteLine(proc[i][3] + "\t\t" + stime[i] + "\t\t" + ctime[i] + "\t\t" + tat[i]
+ "\t\t\t" + wt[i]);

            // Display the process details
            if (i != totalprocess - 1)
            {
                stime[i + 1] = ctime[i];
                ctime[i + 1] = stime[i + 1] + tat[i + 1] - wt[i + 1];
            }
        }

        Console.WriteLine();
        // Display the average waiting time and average turn around time
        Console.WriteLine("Average waiting time is: " + (double)wavg / totalprocess);
        Console.WriteLine("Average turnaround time is: " + (double)tavg / totalprocess);
    }
}
```

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 8

Enter arrival time for process 1: 0
Enter burst time for process 1: 2
Enter priority for process 1: 3

Enter arrival time for process 2: 1
Enter burst time for process 2: 3
Enter priority for process 2: 2

Enter arrival time for process 3: 2
Enter burst time for process 3: 3
Enter priority for process 3: 4

Enter arrival time for process 4: 3
Enter burst time for process 4: 4
Enter priority for process 4: 2

Enter arrival time for process 5: 5
Enter burst time for process 5: 3
Enter priority for process 5: 5

Enter arrival time for process 6: 3
Enter burst time for process 6: 5
Enter priority for process 6: 2

Enter arrival time for process 7: 4
Enter burst time for process 7: 2
Enter priority for process 7: 1
```

```
Enter arrival time for process 8: 4
Enter burst time for process 8: 2
Enter priority for process 8: 1

Process_no      Start_time      Complete_time   Turn_Around_Time        Waiting_Time
1               1               3               2                       0
2               3               6               5                       2
3               6               9               7                       4
4               9               13              10                      6
6               13              18              15                      10
7               18              20              16                      14
8               20              22              18                      16
5               22              25              20                      17

Average waiting time is: 8.625
Average turnaround time is: 11.625
```

Task 2: Perform the above code with 10 Number of processes by randomly setting the arrival time and burst time and priority.
(a) Use Round Robin Algorithm
(b) Use Priority Based Algorithm

(a)

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 10

Enter the arrival times:
Arrival time for p1: 0
Arrival time for p2: 1
Arrival time for p3: 2
Arrival time for p4: 3
Arrival time for p5: 4
Arrival time for p6: 5
Arrival time for p7: 6
Arrival time for p8: 7
Arrival time for p9: 8
Arrival time for p10: 9

Enter the burst times:
Burst time for p1: 2
Burst time for p2: 4
Burst time for p3: 6
Burst time for p4: 8
Burst time for p5: 9
Burst time for p6: 4
Burst time for p7: 2
Burst time for p8: 5
Burst time for p9: 1
Burst time for p10: 4

Enter the quantum time: 3
```

```
process ctime   wtime
 p1     2       0
 p2     14      10
 p3     19      13
 p4     36      28
 p5     38      29
 p6     29      25
 p7     17      15
 p8     37      32
 p9     25      24
 p10    36      32

Average waiting time is 20.8
Average completion time is 25.3
Sequence is like that ->p1->p2->p3->p4->p5->p2->p6->p3->p7->p4->p8->p5->p9->p6->p10->p4->p5->p8->p10
```

(b)

```
faisal152@faisal152:~/schedullingApp$ dotnet run
Enter the number of processes: 10

Enter arrival time for process 1: 0
Enter burst time for process 1: 2
Enter priority for process 1: 3

Enter arrival time for process 2: 2
Enter burst time for process 2: 3
Enter priority for process 2: 4

Enter arrival time for process 3: 1
Enter burst time for process 3: 3
Enter priority for process 3: 4

Enter arrival time for process 4: 4
Enter burst time for process 4: 5
Enter priority for process 4: 3

Enter arrival time for process 5: 4
Enter burst time for process 5: 25
Enter priority for process 5: 7

Enter arrival time for process 6: 4
Enter burst time for process 6: 5
Enter priority for process 6: 6

Enter arrival time for process 7: 4
Enter burst time for process 7: 3
Enter priority for process 7: 8
```

```
Enter arrival time for process 8: 6
Enter burst time for process 8: 4
Enter priority for process 8: 6

Enter arrival time for process 9: 7
Enter burst time for process 9: 8
Enter priority for process 9: 9

Enter arrival time for process 10: 5
Enter burst time for process 10: 4
Enter priority for process 10: 3

Process_no      Start_time      Complete_time   Turn_Around_Time        Waiting_Time
1               1               3               2                       0
3               3               6               5                       2
2               6               9               7                       4
4               9               14              10                      5
6               14              19              15                      10
5               19              44              40                      15
7               44              47              43                      40
10              47              51              46                      42
8               51              55              49                      45
9               55              63              56                      48

Average waiting time is: 21.1
Average turnaround time is: 27.3
```

Lab #10

Task 1: Take two inputs from user and then perform arithmetic operations

```
faisal152@faisal152:~$ vi lab10.sh
```

```
#! /bin/bash

echo -n  "Enter the first number: "
read num1

echo -n "Enter second number: "
read num2

sum=$(expr $num1 + $num2)
difference=$(expr $num1 - $num2)
product=$(expr $num1 \* $num2)
quotient=$(expr $num1 / $num2)
remainder=$(expr $num1 % $num2)

echo "Sum: $sum"
echo "Difference: $difference"
echo "Product: $product"
echo "Quotient: $quotient"
echo "Remainder: $remainder"
```

```
faisal152@faisal152:~$ chmod 755 lab10.sh
```

```
faisal152@faisal152:~$ ./lab10.sh
Enter the first number: 2
Enter second number: 3
Sum: 5
Difference: -1
Product: 6
Quotient: 0
Remainder: 2
```

Task 2: Use system variables and user defined variables to show how that value can be accessed.

```
faisal152@faisal152:~$ vi lab10-2
```

```
#!/bin/bash

echo "System Variables"
echo "User: $USER"
echo "Home Directory: $HOME"
echo "Shell: $SHELL"
echo "Hostname: $HOSTNAME"
echo ""

echo "User-Defined Variables"
var1="Hello World!"
echo "var1: $var1"

num1=10
num2=5
sum=$((num1+num2))
echo "Sum of $num1 and $num2 is: $sum"
echo ""
```

```
echo "Current Date and Time:"
current_datetime=$(date)
echo "$current_datetime"
```

```
faisal152@faisal152:~$ chmod 755 lab10-2
```

```
faisal152@faisal152:~$ ./lab10-2
System Variables
User: faisal152
Home Directory: /home/faisal152
Shell: /bin/bash
Hostname: faisal152

User-Defined Variables
var1: Hello World!
Sum of 10 and 5 is: 15

Current Date and Time:
Wed 26 Jun 2024 06:46:42 AM GMT
```

Task 3: Show results of double , single and back quote

```
faisal152@faisal152:~$ vi lab10-3.sh
```

```
#!/bin/bash

var="Hello World!"
echo "Double quotes: $var"

var='Hello World!'
echo 'Single quotes: $var'

current_date=`date`
echo "Backticks (date command): $current_date"
```

```
faisal152@faisal152:~$ chmod 755 lab10-3.sh
```

```
faisal152@faisal152:~$ ./lab10-3.sh
Double quotes: Hello World!
Single quotes: $var
Backticks (date command): Wed 26 Jun 2024 07:03:26 AM GMT
```

Lab #11

Task 1: Write a program to generate the following output by using while loop

The value of x is 10
The value of x is 9
The value of x is 8
The value of x is 7
The value of x is 6
The value of x is 5
The value of x is 4
The value of x is 3
The value of x is 2
The value of x is 1

```
faisal152@faisal152:~$ vi lab11-1.sh
```

```
#!/bin/bash

x=10

while [$x -gt 0]; do
    echo "The value of x is $x"
    x=$(expr $x - 1)
done
```

```
faisal152@faisal152:~$ ./lab11-1.sh
The value of x is 10
The value of x is 9
The value of x is 8
The value of x is 7
The value of x is 6
The value of x is 5
The value of x is 4
The value of x is 3
The value of x is 2
The value of x is 1
```

Task 2: Write a program to generate the table of any number by using for loop.

```
faisal152@faisal152:~$ vi lab11-2.sh
```

```
#!/bin/bash

echo -n "Enter a number to generate its multiplication table: "
read num

echo "Multiplication table of $num:"
for (( i=1; i<=10; i++ )); do
    result=$(expr $num \* $i)
    echo "$num * $i = $result"
done
```

```
faisal152@faisal152:~$ chmod 755 lab11-2.sh
```

```
Enter a number to generate its multiplication table:
Multiplication table of 3:
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Task 3: Write a program to generate the factorial of a number.

```
faisal152@faisal152:~$ vi lab11-3.sh
```

```
#!/bin/bash

factorial() {
    num=$1
    fact=1

    for (( i = 1; i <= num; i++)); do
        fact=$(expr $fact \* $i)
    done

    echo $fact
}

echo -n "Enter a number find its factorial: "
read num

if [[ $num =~ ^[0-9]+$ ]]; then
   result=$(factorial $num)
   echo "Factorial of $num is: $result"
else
   echo "Error: Please enter a valid non negative integer."
fi
```

```
faisal152@faisal152:~$ chmod 755 lab11-3.sh
```

```
faisal152@faisal152:~$ ./lab11-3.sh
Enter a number find its factorial: 3
Factorial of 3 is: 6
```

```
faisal152@faisal152:~$ ./lab11-3.sh
Enter a number find its factorial: -8
Error: Please enter a valid non negative integer.
```

Task 4: Write a program to show the grades and % of students (with students details).

```
faisal152@faisal152:~$ vi lab11-4.sh
```

```
#!/bin/bash

calculate_percentage(){
   total_marks=$1
   max_marks=$2

   percentage=$(echo "scale=2; ($total_marks / $max_marks) * 100" | bc)
   echo $percentage
}

calculate_grade(){
   percentage=$1

   if (( $(echo "$percentage >= 90" | bc -l) )); then
      echo "A"
   elif (( $(echo "$percentage >= 80" | bc -l) )); then
      echo "B"
   elif (( $(echo "$percentage >= 70" | bc -l) )); then
      echo "C"
   elif (( $(echo "$percentage >= 60" | bc -l) )); then
      echo "D"
   else
      echo "F"
   fi
}
```

```
display_student_details(){
  name=$1
  roll_number=$2
  marks=$3
  max_marks=$4

  percentage=$(calculate_percentage $marks $max_marks)
  grade=$(calculate_grade $percentage)

  echo "Student Name: $name"
  echo "Roll number: $roll_number"
  echo "Marks Obtained: $marks out of $max_marks"
  echo "Percentage: $percentage%"
  echo "Grade: $grade"
  echo
}

students=(
   "Faisal|001|450|500"
   "Farhan|002|400|500"
   "Furqan|003|350|500"
)

for student in "${students[@]}"; do
   IFS='|' read name roll_number marks max_marks <<< "$student"
   display_student_details "$name" "$roll_number" "$marks" "$max_marks"
done
```

```
faisal152@faisal152:~$ chmod 755 lab11-4.sh
```

```
faisal152@faisal152:~$ ./lab11-4.sh
Student Name: Faisal
Roll number: 001
Marks Obtained: 450 out of 500
Percentage: 90.00%
Grade: A

Student Name: Farhan
Roll number: 002
Marks Obtained: 400 out of 500
Percentage: 80.00%
Grade: B

Student Name: Furqan
Roll number: 003
Marks Obtained: 350 out of 500
Percentage: 70.00%
Grade: C
```

Lab #13

Task 1: Write a Bash script that takes a username as an argument and prints the user's ID, home directory, and login shell using switch case.

```
faisal152@faisal152:~$ nano user_info
```

```
#!/bin/bash

# Check if a username was provided
if [ -z "$1" ]; then
    echo "Usage: $0 username"
    exit 1
fi

username=$1

# Check if the user exists
if ! id "$username" &>/dev/null; then
    echo "User '$username' does not exist."
    exit 1
fi

# Get user details
user_id=$(id -u "$username")
home_dir=$(eval echo ~$username)
login_shell=$(getent passwd "$username" | cut -d: -f7)

# Output user details using a case statement
case $1 in
    $username)
        echo "User ID: $user_id"
        echo "Home Directory: $home_dir"
        echo "Login Shell: $login_shell"
        ;;
    *)
        echo "Error: Unexpected case"
        ;;
esac
```

```
faisal152@faisal152:~$ chmod +x user_info
```

```
faisal152@faisal152:~$ ./user_info faisal152
User ID: 1000
Home Directory: /home/faisal152
Login Shell: /bin/bash
```

Task 2: Write a Bash script that takes three arguments: two numbers and an operator (+, -, *, /). The script should perform the calculation and print the result handle by using switch case.

```
faisal152@faisal152:~$ nano arith
```

```bash
#!/bin/bash

# Check if three arguments were provided
if [ "$#" -ne 3 ]; then
    echo "Usage: $0 num1 operator num2"
    exit 1
fi

num1=$1
operator=$2
num2=$3

# Perform the calculation using a case statement
case $operator in
    +)
        result=$(echo "$num1 + $num2" | bc)
        ;;
    -)
        result=$(echo "$num1 - $num2" | bc)
        ;;
    '*')
        result=$(echo "$num1 * $num2" | bc)
        ;;
    /)
        # Check for division by zero
        if [ "$num2" -eq 0 ]; then
            echo "Error: Division by zero"
            exit 1
        fi
        result=$(echo "scale=2; $num1 / $num2" | bc)
        ;;
    *)
        echo "Error: Invalid operator. Use +, -, *, or /"
        exit 1
        ;;
esac

# Print the result
echo "Result: $result"
```

```
faisal152@faisal152:~$ chmod +x arith
faisal152@faisal152:~$ ./arith 5 + 2
Result: 7
faisal152@faisal152:~$ ./arith 6 / 3
Result: 2.00
faisal152@faisal152:~$ ./arith 6 - 3
Result: 3
faisal152@faisal152:~$ ./arith 6 \* 3
Result: 18
```

Task 3: Write a Bash script that counts and prints the number of files in a given directory and its subdirectories handle by using switch case.

```
faisal152@faisal152:~$ nano count_files
```

```
#!/bin/bash

# Check if a directory was provided
if [ -z "$1" ]; then
    echo "Usage: $0 directory"
    exit 1
fi

directory=$1

# Check if the provided argument is a directory
if [ ! -d "$directory" ]; then
    echo "Error: '$directory' is not a directory."
    exit 1
fi

# Count the number of files in the directory and its subdirectories
file_count=$(find "$directory" -type f | wc -l)

# Output the result using a case statement
case $directory in
    *)
        echo "Number of files in '$directory' and its subdirectories: $file_count"
        ;;
esac
```

```
faisal152@faisal152:~$ chmod 755 count_files
faisal152@faisal152:~$ ./count_files /usr/bin
Number of files in '/usr/bin' and its subdirectories: 1209
```

To find Odd or Even no.

```
a=2
b=0
echo Enter the no.
read x
b=`expr $x % $a`
if [ $b -eq 0 ]
then
echo Given no. is Even
else
echo Given no. is Odd
```

To reverse a no.

```
echo Enter the no.
read x
while [$x  -gt 0]
do
c=`expr $x % 10`
echo $c
x=`expr $x/10`
done
```

To print the fibonnicii series

```
count=1
f1=1
f2=1
k=0
echo $f1
echo $f2
```

```
while [ $count  -le 10 ]
do
k=`expr $f1 + $f2`
echo $k
f1=$f2
f2=$k
count =`expr $count +1`
done
```

To find average of two nos.

```
echo enter the 1st no.
read a
echo enter the 2nd no.
read b
c=`expr $a + $b'
d=$c/2
echo the average of two no. is $d
```

Task 4: Implement all the above logic using functions

```
faisal152@faisal152:~$ nano func
```

```
#!/bin/bash

# Function to determine if a number is odd or even
is_odd_or_even() {
    local num=$1
    if [ $(expr $num % 2) -eq 0 ]; then
        echo "Given number is Even"
    else
        echo "Given number is Odd"
    fi
}

# Function to reverse a number
reverse_number() {
    local num=$1
    local rev=0
    while [ $num -gt 0 ]; do
        local digit=$(expr $num % 10)
        rev=$(expr $rev \* 10 + $digit)
        num=$(expr $num / 10)
    done
    echo "Reversed number is $rev"
}

# Function to print the Fibonacci series up to a given count
print_fibonacci() {
    local count=$1
    local f1=1
    local f2=1
    local k=0
    echo $f1
    echo $f2
    for ((i=3; i<=count; i++)); do
        k=$(expr $f1 + $f2)
        echo $k
        f1=$f2
        f2=$k
    done
}
```

```
# Function to find the average of two numbers
average_of_two_numbers() {
    local num1=$1
    local num2=$2
    local sum=$(expr $num1 + $num2)
    local avg=$(echo "scale=2; $sum / 2" | bc)
    echo "The average of $num1 and $num2 is $avg"
}

# Main script

# Prompt for input and call the functions
echo "Enter a number to check if it's odd or even:"
read x
is_odd_or_even $x

echo "Enter a number to reverse:"
read x
reverse_number $x

echo "Enter the count for Fibonacci series:"
read count
print_fibonacci $count

echo "Enter the 1st number to find average:"
read a
echo "Enter the 2nd number to find average:"
read b
average_of_two_numbers $a $b
```

```
faisal152@faisal152:~$ chmod 755 func
```

```
faisal152@faisal152:~$ ./func
Enter a number to check if it's odd or even:
3
Given number is Odd
Enter a number to reverse:
23
Reversed number is 32
Enter the count for Fibonacci series:
2
1
1
Enter the 1st number to find average:
3
Enter the 2nd number to find average:
4
The average of 3 and 4 is 3.50
```

Lab #14

Task 1: Demonstrating Hello World Example
Pre-requisite
Create an account with DockerHub
Open PWD Platform on your browser
Click on Add New Instance on the left side of the screen to bring up Alpine OS instance on the right side

Write command:

$ docker run hello-world